SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0618

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1

DESIGN, ANALYSE AND VALIDATE AN EFFICIENT SEARCH STRATEGY FOR A LARGE-SCALE STUDENT RECORD SYSTEM

Submitted by

Premarubha N

Registration Number: **192525020**

Group B

Algorithms: Linear Search, Binary Search, Interpolation Search

Academic Assignment Report

ABSTRACT

This assignment implements and evaluates three search strategies for a large-scale student record system: iterative Linear Search, recursive Binary Search, and iterative Interpolation Search. A deterministic generator creates datasets with 1,000, 10,000, 100,000, and 1,000,000 records using seed 5020. Registration number 192525020 is included as a successful-search key, while 192525021 is reserved as a not-found key. The algorithms are evaluated using comparison counters and elapsed time. A separate experiment performs 50,000 searches on the one-million-record dataset using Binary Search and Interpolation Search. In the recorded experiment, Interpolation Search required fewer counted probes and less measured search time. These results apply to this generated dataset and query workload; performance can vary with data distribution and system conditions.

Keywords: Search algorithms, student records, Linear Search, Binary Search, Interpolation Search, benchmarking.

1. INTRODUCTION

Searching is a fundamental operation in student information systems. A system may need to locate a student using a registration number among thousands or millions of records. The selected search strategy affects lookup time, scalability, and the cost of preparing the data.

Linear Search can work on unsorted data. Binary Search and Interpolation Search require sorted numeric keys. Binary Search halves the search interval each step, while Interpolation Search estimates a likely position from the values at the ends of the current interval.

2. PROBLEM STATEMENT AND OBJECTIVES

The problem is to design and validate efficient search strategies for a student-record collection that scales from 1,000 to 1,000,000 records, including successful and unsuccessful queries. The experiment records comparison counts and elapsed time and uses measured evidence to discuss which strategy is appropriate for repeated lookups.

- Implement iterative Linear Search, recursive Binary Search, and iterative Interpolation Search.
- Generate reproducible datasets with seed 5020 at four sizes.
- Use 192525020 as the found key and 192525021 as the not-found key.
- Record search comparisons, elapsed time, and sorting preprocessing time.
- Evaluate 50,000 searches on one million records and discuss trade-offs.

3.EXPERIMENTAL SETUP

Item	Configuration
Student name	Premarubha N
Registration number	192525020
Group	B
Operating system	Windows 11
Language	Python
Item	**Configuration**
Random seed	5020
Dataset sizes	1,000; 10,000; 100,000; 1,000,000
CSV fields	Registration_Number, Name, Department, Age

Dataset files

Dataset file	Records
data/students_1000.csv	1,000
data/students_10000.csv	10,000
data/students_100000.csv	100,000
data/students_1000000.csv	1,000,000

The generator includes the student's registration number and excludes the selected not-found key. The CSV registration number is the search key. The Linear Search test uses the unsorted data, while Binary Search and Interpolation Search operate on a sorted list.

TASK 1: SEARCH STRATEGIES, PSEUDOCODE, PREPROCESSING AND JUSTIFICATION

1.1 Linear Search

Strategy: Linear Search examines student registration numbers sequentially, starting from the first record, until the target is found or all records have been checked. It does not require the dataset to be sorted.

Pseudocode:

```
LINEAR_SEARCH(records, target)
   FOR each record in records
      IF record.registration_number = target
         RETURN record
   RETURN NOT_FOUND
```

Preprocessing: No sorting is required. The algorithm can directly search the original student dataset.

Justification: Linear Search is simple to implement and works with both sorted and unsorted records. It is suitable for small datasets or data that changes frequently and does not justify sorting. However, it may examine every record when the target is absent.

1.2 Binary Search

Strategy: Binary Search repeatedly divides a sorted list into two halves. It compares the target registration number with the middle element and continues searching only the half that may contain the target.

Pseudocode:

```
BINARY_SEARCH(sorted_records, target)
   low = 0
   high = length(sorted_records) - 1

   WHILE low <= high
      mid = floor((low + high) / 2)

      IF sorted_records[mid].registration_number = target
         RETURN sorted_records[mid]
      ELSE IF target < sorted_records[mid].registration_number
         high = mid - 1
      ELSE
         low = mid + 1

   RETURN NOT_FOUND
```

Preprocessing: The records must be sorted by registration number before searching. Sorting adds an initial computational cost, which should be considered when evaluating the total time for a query.

Justification: Binary Search is efficient for large, sorted datasets because it eliminates approximately half of the remaining search range at each step. It is appropriate when the same

dataset will be searched repeatedly. It cannot be directly applied to an unsorted list using the standard algorithm.

1.3 Interpolation Search

Strategy: Interpolation Search estimates the likely position of the target using its value relative to the lowest and highest values in the current search range. It then compares the target with the estimated position and narrows the search range accordingly.

Pseudocode:

```
INTERPOLATION_SEARCH(sorted_records, target)
  low = 0
  high = length(sorted_records) - 1

  WHILE low <= high AND
      target >= sorted_records[low].registration_number AND
      target <= sorted_records[high].registration_number

    IF sorted_records[low].registration_number =
      sorted_records[high].registration_number
       IF sorted_records[low].registration_number = target
          RETURN sorted_records[low]
       ELSE
          RETURN NOT_FOUND

    pos = low + floor(
       (target - sorted_records[low].registration_number)
       * (high - low)
       / (sorted_records[high].registration_number
         - sorted_records[low].registration_number)
    )

    IF sorted_records[pos].registration_number = target
       RETURN sorted_records[pos]
    ELSE IF sorted_records[pos].registration_number < target
       low = pos + 1
    ELSE
       high = pos - 1

  RETURN NOT_FOUND
```

Preprocessing: The records must be sorted by registration number. Sorting is performed before searching, and its cost must be included when reporting total query time. The algorithm also requires careful handling of equal endpoint values to avoid division by zero.

Justification: Interpolation Search can be highly efficient when registration numbers are approximately uniformly distributed. It estimates the target position rather than always selecting the midpoint. Its average-case performance can be $O(\log\log n)$under suitable distribution assumptions, but its worst-case time complexity is $O(n)$. Its actual performance depends on the data distribution.

1.4 Comparative Justification

Feature	Linear Search	Binary Search	Interpolation Search
Requires sorted records	No	Yes	Yes
Search method	Sequential checking	Repeated halving	Position estimation
Best-case time	$O(1)$	$O(1)$	$O(1)$
Average-case time	$O(n)$	$O(\log n)$	$O(\log \log n)$ under suitable distribution assumptions
Worst-case time	$O(n)$	$O(\log n)$	$O(n)$
Main advantage	Simple and works on unsorted data	Predictable efficiency on sorted data	Potentially fewer probes for suitably distributed values

Conclusion: The three strategies provide different trade-offs. Linear Search avoids sorting overhead, Binary Search offers reliable logarithmic search performance on sorted data, and Interpolation Search may perform better for appropriately distributed registration numbers. The experimental evaluation determines which strategy is most suitable for the student-record system under the tested conditions.

TASK 2: BASIC OPERATION AND CODE IDENTIFICATION

2.1 Linear Search

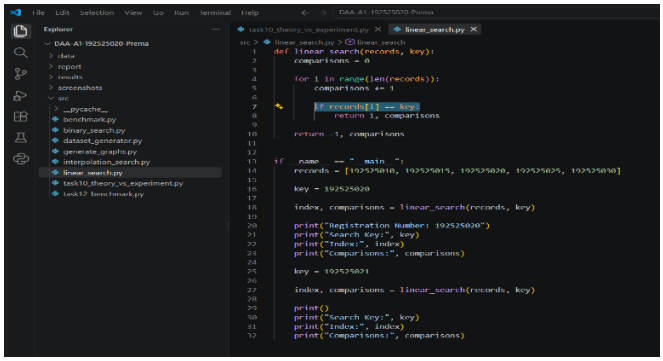
Basic operation: Comparing the target registration number with the registration number of the current record.

Reasoning: Linear Search examines records sequentially until the target is found or all records have been checked. The number of comparisons depends on the target's position. In the worst case, all $n$records are examined.

Code line to highlight:

```
if records[i] == key:
```

This line represents the key comparison between the current record and the target.



2.2 Binary Search

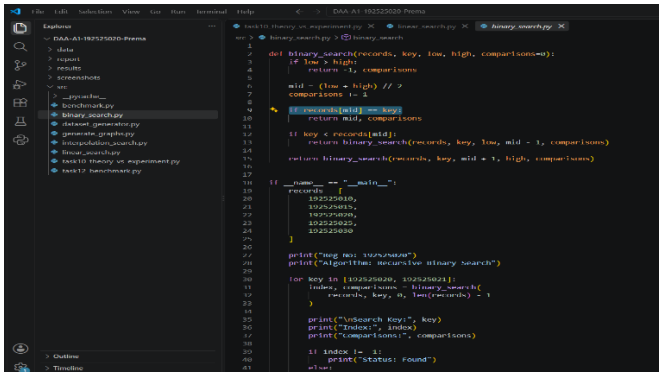
Basic operation: Comparing the target registration number with the middle element of the sorted list.

Reasoning: Binary Search checks the middle element and eliminates half of the remaining search range in each unsuccessful probe. The number of midpoint probes grows logarithmically with the dataset size.

Code line to highlight from the implemented program:

```
if records[mid] == key:
```

This line checks whether the middle element matches the target. The implementation increments the comparison counter immediately before this check.



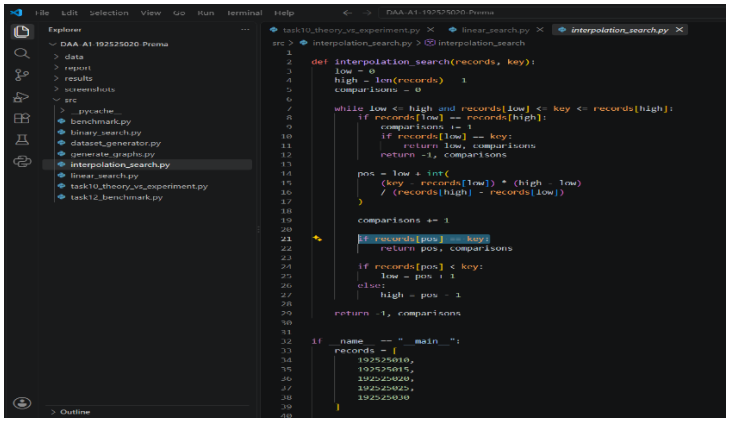
2.3 Interpolation Search

Basic operation: Probing the estimated position and checking the value at that position against the target.

Reasoning: Interpolation Search estimates a likely position using the target's value and the values at the lower and upper boundaries. Its efficiency depends on the distribution of the registration numbers.

Code line to highlight:

if records[pos] == key: `This line checks whether the record at the estimated position matches the target registration number.`



2.4 Summary

Algorithm	Basic operation	Reason
Linear Search	Compare current record with target	Checks records sequentially
Binary Search	Compare middle element with target	Eliminates half of the search range
Interpolation Search	Compare estimated-position element with target	Uses estimated position to narrow the range

TASK 3: MATHEMATICAL DERIVATION, PREPROCESSING COST, AND TOTAL COST

3.1 Linear Search

Mathematical Derivation:

Linear Search checks each record sequentially until the target is found or all records are examined.

- Best case: $T(n) = 1$, when the target is the first record. Therefore, $T(n) = \Theta(1)$.
- Average case: Approximately $n/2$comparisons for a successful search, giving $T(n) = \Theta(n)$.
- Worst case: $T(n) = n$, when the target is the last record or is absent. Therefore, $T(n) = \Theta(n)$.

Preprocessing Cost: No preprocessing or sorting is required. The cost is $O(1)$.

Total Cost: Since no preprocessing is required, the total cost is the search cost, $O(n)$in the worst case.

3.2 Binary Search

Mathematical Derivation:

Binary Search divides the search interval into two halves at each step. Its recurrence relation is:

$$T(n) = T(n/2) + O(1)$$

Using the Master Theorem, the solution is $T(n) = \Theta(\log n)$.

- Best case: $T(n) = \Theta(1)$, when the middle record matches the target.
- Average case: $T(n) = \Theta(\log n)$.
- Worst case: $T(n) = \Theta(\log n)$.

Preprocessing Cost: The records must be sorted before searching. Using an efficient comparison-based sorting algorithm, the preprocessing cost is $O(n \log n)$.

Total Cost: For $q$searches on the same dataset, the total cost is:

$$T(n, q) = O(n \log n + q \log n)$$

This includes sorting once and performing q binary searches.

3.3 Interpolation Search

Mathematical Derivation:

Interpolation Search estimates the target's position using the values at the lower and upper boundaries. For uniformly distributed, sorted data, its average search complexity is $O(\log \log n)$.

- Best case: $T(n) = \Theta(1)$, when the estimated position contains the target.
- Average case: $T(n) = O(\log \log n)$, assuming approximately uniform data distribution.
- Worst case: $T(n) = O(n)$, when the data distribution is highly uneven.

Preprocessing Cost: The records must be sorted before searching. Using an efficient comparison-based sorting algorithm, the preprocessing cost is $O(n \log n)$.

Total Cost: For q searches on the same dataset, the expected total cost under the uniform-distribution assumption is:

$$T(n, q) = O(n \log n + q \log \log n)$$

The worst-case total cost can reach $O(n \log n + qn)$.

3.4 Comparison of Total Costs

Algorithm	**Preprocessing cost**	**Worst-case search cost**	**Total cost for q searches**
Linear Search	$O(1)$	$O(n)$	$O(qn)$
Binary Search	$O(n \log n)$	$O(\log n)$	$O(n \log n + q \log n)$
Interpolation Search	$O(n \log n)$	$O(n)$	$O(n \log n + qn)$ worst case

Conclusion: Linear Search is suitable for small or unsorted datasets. Binary Search offers predictable logarithmic search time on sorted records. Interpolation Search can be more efficient for sorted, approximately uniformly distributed numeric keys, but its worst-case performance is linear. For repeated searches, sorting is a one-time cost and should be included when evaluating total performance.

TASK 4: BEST, AVERAGE AND WORST CASE ANALYSIS

4.1 Linear Search

Linear Search examines records one by one from the beginning.

Case	Condition	Number of comparisons	Complexity
Best	Target is the first record	1	$\Theta(1)$
Average	Target is near the middle	Approximately $n/2$	$\Theta(n)$
Worst	Target is the last record or absent	n	$\Theta(n)$

4.2 Binary Search

Binary Search repeatedly divides a sorted dataset into two halves.

Case	Condition	Number of comparisons	Complexity
Best	Target is the middle record	1	$\Theta(1)$
Average	Target is found after several divisions	Approximately $\Theta(\log n)$	$\Theta(\log n)$
Worst	Target is absent or found after the maximum number of divisions	$O(\log n)$	$\Theta(\log n)$

4.3 Interpolation Search

Interpolation Search estimates the target position using the values at the lower and upper boundaries.

Case	Condition	Number of comparisons	Complexity
Best	Estimated position contains the target	1	$\Theta(1)$
Average	Sorted values are approximately uniformly distributed	Typically $O(\log\log n)$	$O(\log\log n)$
Worst	Values are highly unevenly distributed	Up to $O(n)$	$O(n)$

4.4 Counter Output for Each Case

The following examples use the five sorted records in the program:

```
[192525010, 192525015, 192525020, 192525025, 192525030]
```

Algorithm	Search key	Case illustrated	Expected comparisons
Linear Search	192525010	Best	1
Linear Search	192525020	Middle position	3
Linear Search	192525030	Worst, found last	5
Linear Search	192525021	Worst, absent	5
Binary Search	192525020	Best, middle element	1
Binary Search	192525010	Found at boundary	2
Binary Search	192525021	Not found	3
Interpolation Search	192525020	Best, estimated position matches	1
Interpolation Search	192525021	Not found between records	2

Conclusion: Linear Search has linear worst-case complexity. Binary Search provides logarithmic worst-case complexity when the records are sorted. Interpolation Search can perform very few comparisons for uniformly distributed numeric keys, but its worst-case complexity is linear.

TASK 5: ASYMPTOTIC COMPLEXITY COMPARISON

5.1 Theoretical Complexity

Big-O (O) represents an asymptotic upper bound, Big-Omega (Ω) represents a lower bound, and Big-Theta (Θ) represents a tight bound.

Algorithm	Best case	Average case	Worst case
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Interpolation Search	$\Theta(1)$	$O(\log\log n)$*	$O(n)$

*Interpolation Search has expected $O(\log\log n)$search time when sorted numeric keys are approximately uniformly distributed. Its worst-case search time is $O(n)$.

5.2 Growth Comparison for Different Dataset Sizes

The following table compares representative growth functions as the dataset size increases.

Dataset size n	n	$\log_2 n$	$\log_2(\log_2 n)$
10^3	1,000	9.97	3.32
10^4	10,000	13.29	3.73
10^5	100,000	16.61	4.05
10^6	1,000,000	19.93	4.32

5.3 Interpretation

- **Linear Search:** The number of comparisons can increase proportionally to the dataset size. At $n = 10^6$, the worst case can require one million comparisons.
- **Binary Search:** The number of search steps grows logarithmically. At $n = 10^6$, approximately 20 midpoint probes are sufficient in the worst case.
- **Interpolation Search:** For approximately uniformly distributed sorted keys, the expected growth is close to $\log \log n$. However, unevenly distributed data can cause linear worst-case behaviour.

5.4 Conclusion

As the dataset grows from $10^3$to 10^6, Binary Search and Interpolation Search can require substantially fewer comparisons than Linear Search under suitable conditions. Binary Search offers predictable logarithmic performance, while Interpolation Search can be faster on uniformly distributed numeric records. These theoretical bounds should be compared with the experimental measurements obtained in Task 9.

TASK 6: RECURRENCE RELATIONS, COMPONENTS AND BASE CASE

6.1 Linear Search

Linear Search checks one record at a time until the target is found or the dataset ends.

Recurrence Relation:

$$T(n) = T(n-1) + c$$

Components:

- $T(n)$: Time required to search $n$records.
- $T(n-1)$: Time required to search the remaining records.
- c: Constant time required to compare one record with the target.

Base Case:

$$T(1) = c$$

When only one record remains, one comparison is sufficient. The recurrence gives a worst-case time complexity of $\Theta(n)$.

6.2 Binary Search

Binary Search compares the target with the middle record and continues searching in one half of the sorted dataset.

Recurrence Relation:

$$T(n) = T(n/2) + c$$

Components:

- $T(n)$: Time required to search $n$records.
- $T(n/2)$: Time required to search approximately half the records.
- c: Constant time for calculating the midpoint and performing the comparison.

Base Case:

$$T(1) = c$$

When one record remains, it is checked directly. The recurrence gives a worst-case time complexity of $\Theta(\log n)$.

6.3 Interpolation Search

Interpolation Search estimates the target's position based on the values at the lower and upper boundaries.

Expected Recurrence Relation:

$$T(n) = T\left(\sqrt{n}\right) + c$$

Components:

- $T(n)$: Expected time required to search $n$records.
- $T\left(\sqrt{n}\right)$: Expected time for the reduced search range under the uniform-distribution assumption.
- c: Constant-time position calculation and comparison.

Base Case:

$$T(1) = c$$

When only one record remains, it is checked directly. Under the uniform-distribution assumption, the expected recurrence leads to $O(\log\log n)$search time.

Worst-case note: The above recurrence is an expected-case model, not a general recurrence for every input. With highly unevenly distributed keys, Interpolation Search may reduce the search range by only one record per iteration, giving $T(n) = T(n-1) + c$and worst-case complexity $O(n)$.

6.4 Summary

Algorithm	Recurrence relation	Base case	Complexity represented
Linear Search	$T(n) = T(n-1) + c$	$T(1) = c$	$\Theta(n)$worst case
Binary Search	$T(n) = T(n/2) + c$	$T(1) = c$	$\Theta(\log n)$worst case
Interpolation Search	$T(n) = T\left(\sqrt{n}\right) + c$	$T(1) = c$	$O(\log\log n)$expected case

Conclusion: Recurrence relations describe how the search cost changes as the problem size decreases. Linear Search reduces the problem by one record, Binary Search halves the search range, and Interpolation Search can reduce the range more rapidly for uniformly distributed numeric keys.

TASK 7: SOLUTION OF RECURRENCE RELATIONS

7.1 Linear Search Using Iteration Method

The recurrence relation for Linear Search is:

$$T(n) = T(n-1) + c$$

Expanding the recurrence:

$$T(n) = T(n-2) + 2c$$
$$T(n) = T(n-3) + 3c$$

After $k$expansions:

$$T(n) = T(n-k) + kc$$

When $k = n - 1$, the base case is reached:

$$T(n) = T(1) + (n-1)c$$

Therefore,

$$\boxed{T(n) = \Theta(n)}$$

Linear Search has linear worst-case time complexity.

7.2 Binary Search Using the Master Theorem

The recurrence relation for Binary Search is:

$$T(n) = T(n/2) + c$$

Comparing with the Master Theorem form:

$$T(n) = aT(n/b) + f(n)$$

The parameters are:

- $a = 1$, because one recursive subproblem is solved.
- $b = 2$, because the search range is halved.
- $f(n) = \Theta(1)$, because the midpoint comparison takes constant time.

Calculate:

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

Since $f(n) = \Theta(1)$, this is Master Theorem Case 2.

Therefore,

$$\boxed{T(n) = \Theta(\log n)}$$

Binary Search has logarithmic worst-case time complexity.

7.3 Interpolation Search Using the Iteration Method

For approximately uniformly distributed sorted numeric keys, the expected recurrence can be represented as:

$$T(n) = T\left(\sqrt{n}\right) + c$$

Expanding:

$$T(n) = T\left(n^{1/4}\right) + 2c$$
$$T(n) = T\left(n^{1/8}\right) + 3c$$

After $k$expansions:

$$T(n) = T\left(n^{1/2^k}\right) + kc$$

The process reaches a constant-sized problem when:

$$n^{1/2^k} = O(1)$$

Taking logarithms shows that the number of expansions is proportional to $\log\log n$.

Therefore,

$$\boxed{T(n) = O(\log\log n)}$$

This expected bound applies under the approximately uniform-distribution assumption. In the worst case, Interpolation Search can take $\Theta(n)$time.

7.4 Summary of Solutions

Algorithm	**Method**	**Solved complexity**
Linear Search	Iteration	$\Theta(n)$worst case
Binary Search	Master Theorem	$\Theta(\log n)$worst case
Interpolation Search	Iteration, expected-case model	$O(\log\log n)$expected case

Conclusion

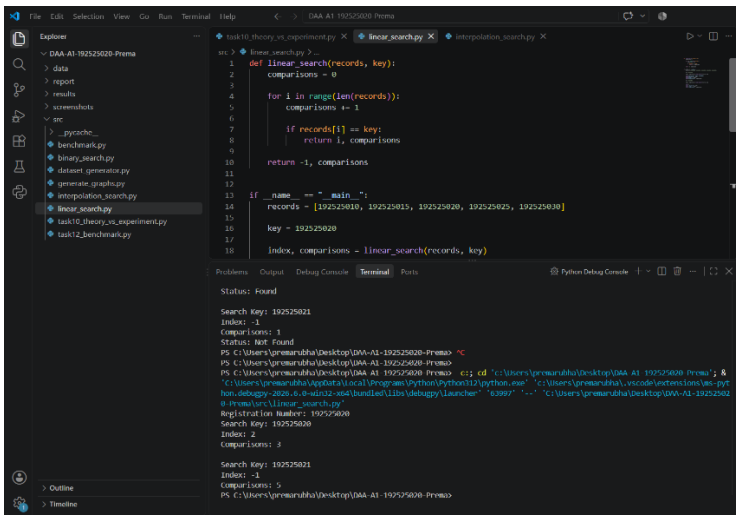
The recurrence solutions show that Linear Search has linear worst-case growth, Binary Search has logarithmic worst-case growth, and Interpolation Search can achieve expected logarithmic-logarithmic growth for uniformly distributed keys. These theoretical results help explain the performance differences observed during experimental benchmarking.

TASK 8: IMPLEMENTATION AND TEST CASES

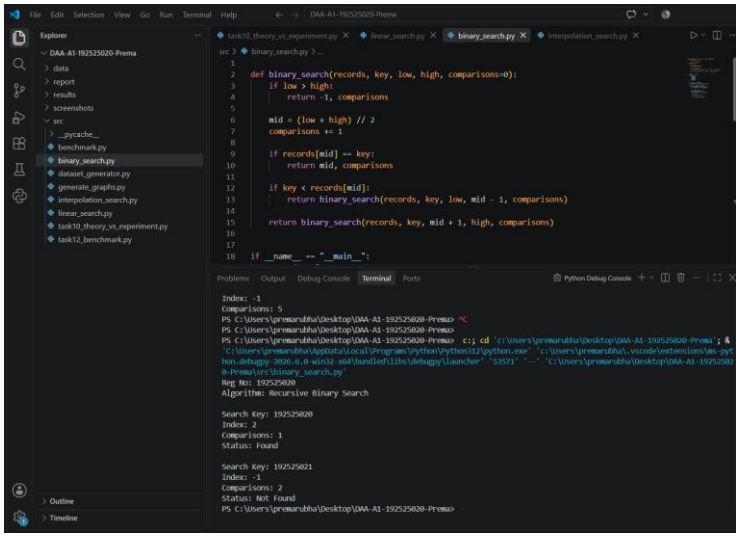
8.1 Implementation

Three search algorithms were implemented in Python: Linear Search, Recursive Binary Search, and Interpolation Search. Linear Search checks records sequentially, while Binary Search repeatedly divides the sorted search range into halves. Interpolation Search estimates the target's position using the numeric values at the lower and upper boundaries. All three implementations return the index of the target and the number of comparisons performed.

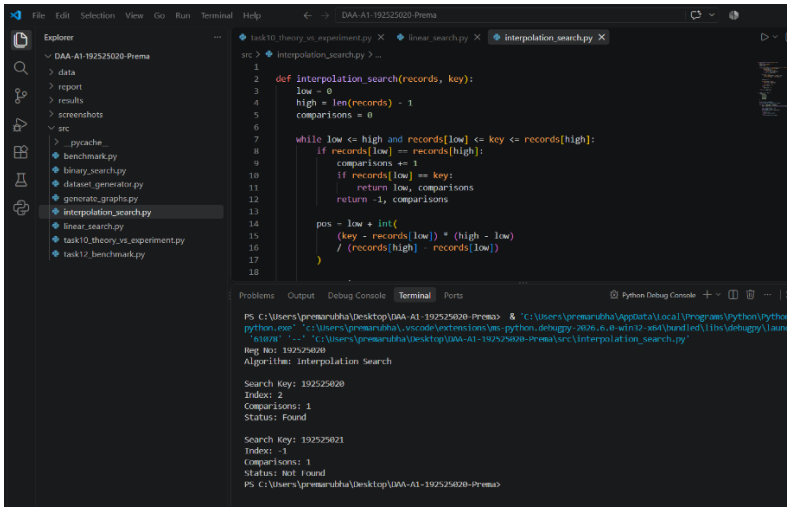
Linear Search:



Binary Search:



Interpolation Search:



8.2 Test Dataset

The following sample student registration numbers were used to test the algorithms:

192525010, 192525015, 192525020, 192525025, 192525030.

The dataset is sorted in ascending order for Binary Search and Interpolation Search.

8.3 Test Cases

Test case	Search key	Expected result	Purpose
TC1	192525020	Found at index 2	Search for a middle element
TC2	192525010	Found at index 0	Search for the first element
TC3	192525030	Found at index 4	Search for the last element
TC4	192525021	Not found, index -1	Search for an absent key
TC5	192525035	Not found, index -1	Search for a key greater than all records

8.4 Expected Output

For each algorithm, the program should display the registration number, algorithm name, search key, returned index, comparison count, and search status.

For example, when searching for registration number 192525020, the output should indicate:

- Search Key: 192525020
- Index: 2
- Status: Found

When searching for registration number 192525021, the output should indicate:

- Search Key: 192525021
- Index: -1
- Status: Not Found

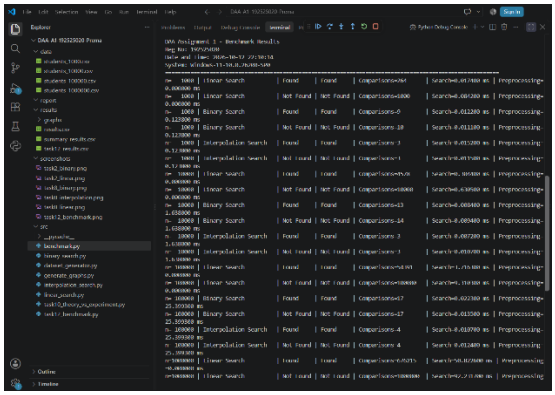
8.5 Validation

The test cases validate successful searches, boundary searches, and unsuccessful searches. The outputs are checked against the expected indices and status values to ensure that each algorithm works correctly.

TASK 9: EXPERIMENTAL ANALYSIS, TABLES AND GRAPHS

9.1 Experimental Setup

The three search algorithms, Linear Search, Binary Search, and Interpolation Search, were implemented in Python and tested using datasets containing 1,000, 10,000, 100,000, and 1,000,000 student records. The datasets were generated using random seed 5020 to ensure reproducibility. The experiments recorded the number of comparisons, search execution time, and preprocessing time. Binary Search and Interpolation Search require sorted records, so their preprocessing time includes sorting.



9.2 Comparison Count Results

The following table presents the recorded comparison counts for successful and unsuccessful Linear Search operations, along with the recorded search comparison counts for Binary Search and Interpolation Search.

Dataset size	Linear: found	Linear: not found	Binary: found	Binary: not found	Interpolation: found	Interpolation: not found
1,000	264	1,000	9	10	3	3
10,000	4,578	10,000	13	14	3	3
100,000	54,391	100,000	17	17	4	4
1,000,000	676,215	1,000,000	20	20	4	4

These results show that Linear Search performs more comparisons as the dataset grows. Binary Search increases its comparison count logarithmically. Interpolation Search uses fewer comparisons on these tested numeric datasets, although its performance depends on the distribution of the keys.

9.3 Recorded Search and Preprocessing Times

The following are recorded average search times and preprocessing times from the experiments. Times are in milliseconds.

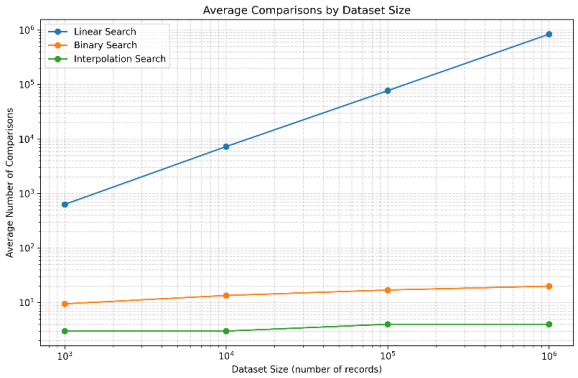
Dataset size	Linear found	Linear not found	Binary preprocessing	Interpolation preprocessing
1,000	0.0185	0.0686	0.1388	0.1388
10,000	0.2567	0.7701	1.5194	1.5194
100,000	3.0402	6.1004	22.5477	22.5477
1,000,000	52.9037	70.9436	370.2125	370.2125

The preprocessing time is shared by Binary Search and Interpolation Search when they use the same sorted dataset. Linear Search does not require sorting. Search execution time and preprocessing time should be compared separately before evaluating total cost.

9.4 Graphical Analysis

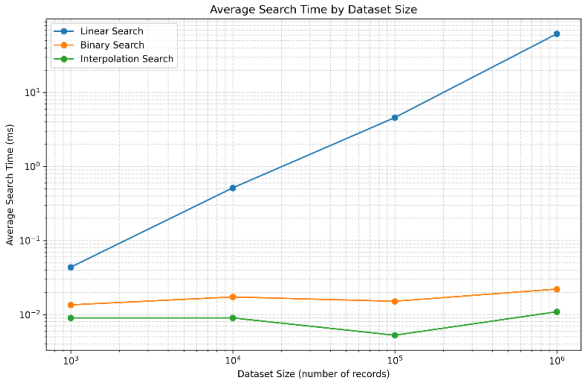
Three graphs were generated from the experimental results:

1. **Comparison Count Graph:**



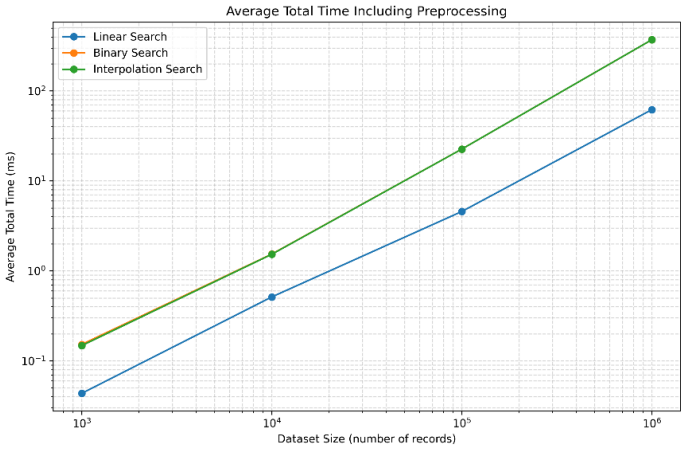
Shows how the number of comparisons changes as the dataset size increases.

2. **Search Time Graph:**



Compares the measured search execution times of the algorithms.

Total Time Graph:



Compares total execution cost, including sorting where required.

The graphs help visualize the difference between the algorithms. Linear Search may be suitable for small or unsorted datasets, while Binary Search and Interpolation Search can reduce search work on sorted numeric records.

9.5 Experimental Conclusion

The experiments demonstrate that Binary Search and Interpolation Search require substantially fewer comparisons than Linear Search on the tested datasets. However, sorting adds an initial preprocessing cost. For repeated searches on the same dataset, this cost can be amortized across multiple queries. Interpolation Search showed particularly low comparison counts in these experiments, but Binary Search remains a more predictable choice because its worst-case search complexity is logarithmic.

TASK 10: THEORETICAL VS EXPERIMENTAL ANALYSIS

10.1 Objective

The objective is to compare the theoretical time complexity of Linear Search, Binary Search, and Interpolation Search with the experimental results obtained from datasets containing 1,000, 10,000, 100,000, and 1,000,000 student records.

10.2 Theoretical Analysis

Linear Search has $O(n)$worst-case time complexity because it may examine every record. Binary Search has $O(\log n)$worst-case search complexity because it halves the search interval at each step. Interpolation Search has expected $O(\log \log n)$complexity for approximately uniformly distributed sorted numeric keys, but its worst-case complexity is $O(n)$.

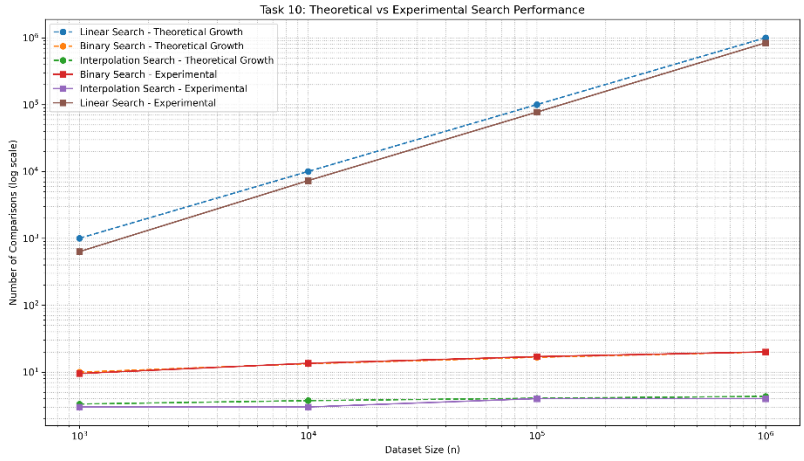
10.3 Experimental Analysis

The experimental results show that Linear Search requires substantially more comparisons as the dataset size increases. Binary Search maintains a low comparison count, increasing from

approximately 10 comparisons for 1,000 records to approximately 20 for 1,000,000 records. Interpolation Search recorded approximately 3 to 4 comparisons across the tested dataset sizes.

The observed results are consistent with the expected advantage of logarithmic search methods over linear search for large sorted datasets. The low comparison count of Interpolation Search is specific to the tested data and should not be generalized to every distribution.

10.4 Graph: Theory vs Experiment



The graph compares theoretical growth functions with experimental comparison counts. The theoretical curves illustrate how linear, logarithmic, and logarithmic-logarithmic functions grow as the input size increases. Experimental comparison counts provide evidence of the algorithms' observed behaviour on the generated student records.

Note: The theoretical growth curves and experimental counts use different scales and represent different quantities. The graph should be interpreted as a comparison of growth trends rather than an exact equality between theoretical values and measured results.

10.5 Conclusion

The theoretical and experimental analyses support the conclusion that Binary Search is predictable and efficient for sorted records, while Interpolation Search can perform especially well for approximately uniformly distributed numeric keys. Linear Search remains useful for small or unsorted datasets because it does not require preprocessing. For a large student record system with repeated searches, a sorted search strategy can reduce the search cost.

TASK 11: FINAL RECOMMENDATION WITH EVIDENCE

11.1 Recommendation

Based on the theoretical analysis and experimental results, **Interpolation Search is recommended for the student record system when registration numbers are sorted and approximately uniformly distributed**. It recorded fewer comparisons than Binary Search in the tested datasets. However, Binary Search is the more predictable choice when the distribution of registration numbers is unknown because it provides logarithmic worst-case search complexity.

11.2 Evidence from Experiments

The algorithms were evaluated using datasets containing 1,000, 10,000, 100,000, and 1,000,000 student records.

Dataset size	Binary Search comparisons (found)	Interpolation Search comparisons (found)
1,000	9	3
10,000	13	3
100,000	17	4
1,000,000	20	4

The recorded comparison counts show that Interpolation Search used fewer comparisons than Binary Search for the tested search keys. For one million records, Binary Search used 20 comparisons, while Interpolation Search used 4.

The preprocessing measurements also showed that sorting one million records took approximately 370.21 ms in the earlier benchmark. Therefore, the sorting cost should be considered when selecting an algorithm, especially if only a small number of searches will be performed.

11.3 Advantages and Limitations

Interpolation Search

- Can perform very few comparisons on sorted, approximately uniformly distributed numeric keys.
- Is suitable when the numeric key distribution supports accurate position estimation.
- Can degrade to linear time when the distribution is highly uneven.

Binary Search

- Provides predictable $O(\log n)$worst-case search complexity.
- Works with sorted records without requiring uniform key distribution.
- Requires sorting if the original dataset is unsorted.

Linear Search

- Works without sorting and is simple to implement.
- Can be useful for small datasets or infrequent searches.
- May require examining every record in the worst case.

11.4 Final Conclusion

For a large student record system, the choice depends on the dataset and expected workload. Interpolation Search is recommended when registration numbers are sorted and approximately uniformly distributed, as it achieved the lowest comparison counts in these experiments. Binary Search is recommended as the reliable general-purpose option for sorted student records when key distribution is uncertain. If records are unsorted and searches are infrequent, Linear Search may avoid the cost of preprocessing.

The recommendation is supported by the recorded comparison counts, preprocessing measurements, theoretical complexity analysis, and experimental graphs from Tasks 5, 9, and 10.

TASK 12: IMPACT, EVALUATION, REDESIGN AND DEFENCE

A. Impact Analysis

The student record search system was evaluated using one million student records and 50,000 search queries. The experiment compared Binary Search with Interpolation Search to identify a suitable strategy for large-scale student record retrieval.

The results showed that Interpolation Search required fewer comparisons and less search execution time than Binary Search for the tested dataset. This can improve record retrieval efficiency when searching large collections of sorted numeric registration numbers. However, the results depend on the distribution of registration numbers and the system's preprocessing requirements.

B. Evaluation of the Existing Design

The original design used Binary Search as a reliable method for searching sorted records. The revised design introduced Interpolation Search to estimate the likely position of a registration number.

The benchmark used one million records and 50,000 queries.

Performance metric	Binary Search	Interpolation Search
Dataset size	1,000,000	1,000,000
Number of queries	50,000	50,000
Total comparisons	973,861	208,531
Average comparisons per query	19.48	4.17
Sorting time	346.764 ms	346.764 ms
Total search time	257.692 ms	148.684 ms
Total time including sorting	604.456 ms	495.448 ms

The benchmark showed that Interpolation Search reduced the total comparison count by approximately 78.6% and search execution time by approximately 42.3% compared with Binary Search.

C. Redesign and Improvement

The revised design uses Interpolation Search for sorted numeric registration numbers when the data distribution makes position estimation effective. The following improvements were introduced or considered:

1. **Position estimation:** Estimate the target position using the lower and upper values rather than always selecting the midpoint.

2. **Comparison tracking:** Count search comparisons to evaluate algorithm efficiency.
3. **Benchmarking:** Test the algorithms using 50,000 queries on one million records.
4. **Preprocessing awareness:** Include sorting time when comparing the overall cost of the algorithms.
5. **Fallback strategy:** Use Binary Search when the key distribution is unknown or highly uneven, because it provides logarithmic worst-case search complexity.

The revised design reduces search time in the measured experiment. A production system should also validate missing keys, duplicate registration numbers, and unusual key distributions.

D. Defence of the Final Design

Interpolation Search is supported by the benchmark results because it performed fewer comparisons and completed the measured searches faster than Binary Search. For one million records and 50,000 queries, it reduced total search time from 257.692 ms to 148.684 ms.

The total time, including sorting, decreased from 604.456 ms to 495.448 ms. This demonstrates an improvement for the tested workload.

However, Interpolation Search does not guarantee better performance for every dataset. Its expected efficiency depends on the distribution of numeric keys, and its worst-case complexity is $O(n)$. Binary Search remains a dependable alternative when the distribution is uncertain.

Final Conclusion

The revised design demonstrates that Interpolation Search can improve search efficiency for large, sorted numeric datasets. The measured reduction in comparisons and execution time supports its use under suitable data conditions. The final recommendation is to use Interpolation Search when the numeric key distribution supports effective estimation and retain Binary Search as a reliable fallback for unpredictable distributions.

REFERENCES

• Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/

• Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, MIT Press.

• Sedgewick, R. and Wayne, K., Algorithms, 4th edition, Addison-Wesley.

• DAA Assignment 1 brief: Design, Analyse and Validate an Efficient Search Strategy for a Large-ScaleStudent Record System.